COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 | **SECTION:** | CPE21S4 |
| **DATE PERFORMED:** | September 4, 2026 | **DATE SUBMITTED:** | September 15, 2026 |
| **NAME:** | Uchi, Mao Sophie S. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO. 7.1** **Sorting Algorithms Pt.1**

## 1. PROCEDURE OUTPUT

**ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.**

**Table 7-1. Array of Values for Sort Algorithm Testing**

**Code + Console Screenshot**

main.cpp sortingAlgo.h

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int max_size = 100;
void displayArr(int arr[]);

int main() {

    // create an array with unsorted list with 100 maximum size
    int unsortedArr[max_size];
    // use time (0) as seed for random values
    srand(time(0));
    // populate the array with random values
    for(int i = 0; i < max_size; i++){
        unsortedArr[i] = rand()%100;
    }
    displayArr(unsortedArr);
```

```cpp
void displayArr(int arr[]){
    for (int i = 0; i < max_size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```
74 36 72 50 8 7 22 0 43 81 0 34 18 74 13 62 80 80 17 48
61 0 46 18 11 34 91 52 89 68 3 65 16 6 1 58 93 27 79 41
63 3 1 37 49 46 71 88 29 48 89 52 82 2 10 87 2 12 43 97
29 31 9 88 90 52 46 50 67 45 87 40 80 8 6 30 38 74 75 70
 34 18 80 58 87 38 93 74 52 85 79 61 59 45 16 95 16 79 9
2 59
PS D:\Downloads (HDD)\CPE010\HOA7>
```

| | |
|---|---|
| **Observations** | What I've observed from these blocks of codes, using the Time Header library, we can access random numbers, but why srand instead of rand, where we can also generate random numbers; the difference would be that rand would output the same random numbers as your program starts at a default seed while srand generates different seeds, hence outputs random numbers that aren't repetitive. |

**Table 7-2. Bubble Sort Technique**

**Code + Console Screenshot**

```
21      std::cout<<"\n Bubble Sort Testing: \n";
22      bubbleSort(unsortedArr, max_size);
23      displayArr(unsortedArr);
```

sortingAlgo.h > ...

```
1   #ifndef SORTINGALGO_H
2   #define SORTINGALGO_H
3   #include <iostream>
4
5   template<typename T>
6   void bubbleSort(T arr[], int max_size){
7       for(int i = 0; i<max_size; i++){
8           for(int j = i+1; j < max_size; j++){
9               // arr[i] // represent the first element
10              if(arr[i]>arr[j]){
11                  std::swap(arr[j],arr[i]);
12              }
13          }
14      }
15  }
```

```
 Bubble Sort Testing:
2 2 2 3 3 5 7 8 10 11 11 11 12 12 12 14 14 14 17 17 18 1
8 18 18 19 19 20 21 23 26 26 28 29 29 30 32 33 34 35 35
39 40 41 42 42 42 44 44 44 45 46 47 48 48 51 54 55 56 57
 58 60 61 64 66 66 67 68 68 69 71 71 71 71 71 72 72 73 7
3 75 76 77 78 80 81 82 82 82 83 84 88 89 89 90 90 91 92
92 96 98 99
PS D:\Downloads (HDD)\CPE010\HOA7>
```

**Observations**

Calling header bubbleSort using the srand's random numbers as base for Bubble Sorting; the random generated numericals are currently unsorted, by comparing the first element (arr[i]) and second element, we must calculate whether the first element is greater than the second element (arr[j]), if the condition is met, the program will swap the 2 elements; if the condition is not met, the program will continue to loop and iterate until there are greater elements needs to be swapped.

**Table 7-3. Selection Sort Algorithm**

**Code + Console Screenshot**

```
25      std::cout<<"\n Selection Sort Testing: \n";
26      selectionSort(unsortedArr, max_size);
27      displayArr(unsortedArr);
```

```
17  template<typename T>
18  int routineSmallest(T arr[], int K, int max_size){
19      int position;
20      T smallestElem = arr[K];
21      position = K;
22
23      for(int j=K+1; j < max_size; j++){
24          if(arr[j] < smallestElem){
25              smallestElem = arr[j];
26              position = j;
27          }
28      }
29      return position;
30  }
```

```
31
32  template<typename T>
33  void selectionSort(T arr[], int max_size){
34      int POS;
35      for(int i = 0; i < max_size; i++){
36          POS=routineSmallest(arr, i , max_size);
37          std::swap(arr[i], arr[POS]);
38      }
39  }
```

```
 Selection Sort Testing:
1 1 3 3 4 5 5 6 6 8 9 10 13 13 13 14 17 17 18 18 19 20 2
1 22 22 23 25 26 26 28 28 32 34 34 35 37 39 40 40 42 42
47 47 48 48 50 50 50 50 50 51 52 53 54 54 56 57 59 60 61
 61 62 62 64 64 69 70 70 71 71 71 73 74 75 75 76 77 77 7
7 77 78 83 83 83 84 85 86 89 89 90 90 91 93 95 96 96 97
97 97 97
PS D:\Downloads (HDD)\CPE010\HOA7>
```

**Observations**

Calling header selectionSort using the srand's random numbers as base for Selection Sorting; the random generated numericals are currently unsorted, by comparing the first element (arr[i]) to the smallest element in the array (arr[POS]); with our template function "routineSmallest" we can find the smallest element by comparing elements that are lesser than its current position. We iterate the smallest element position to our array list, to find our actual smallest element; if the smallest element is found, we can swap the first element and smallest element. The program will continue to loop and iterate until there are smaller elements that need to be swapped.

**Table 7-4. Insertion Sort Algorithm**

**Code + Console Screenshot**

```
16      for(int i = 0; i < max_size; i++){
17          unsortedArr[i] = rand()%5 +1;
```

```
29      std::cout<<"\n Insertion Sort Testing: \n";
30      insertionSort(unsortedArr, max_size);
31      displayArr(unsortedArr);
```

```cpp
template<typename T>
void insertionSort(T arr[], int max_size){
    int k=1, j, temp;

    while(k < max_size){
        temp = arr[k];
        j = k-1;

        while(temp <= arr[j]){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        k++;
    }
}
```

```
 Insertion Sort Testing:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
PS D:\Downloads (HDD)\CPE010\HOA7>
```

| | |
|---|---|
| **Observations** | Calling header insertionSort using the srand's random numbers as base for Insertion Sorting; the random generated numericals are currently unsorted, by assuming the first element as sorted, we will compare the first element to our next element whether they're greater or lesser and place the element in a numerical order from the array. Moving on to our next element, we will compare the current element from our previous 2 elements whether they're greater or lesser and place the element in a numerical order from the array. The program will continue to loop and iterate until there are greater elements that need to be swapped. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO B: Solve given data sorting problems using appropriate basic sorting algorithms**

**Pseudocode of Algorithm**

ilo_b.cpp (main.cpp)

```
START
  DECLARE max_size
  SET max_size to CONST
  SET max_size to 100

  DECLARE candidateL with an array size of 5
  SET candidateL equals to { "Bo Dalton Capistrano",
  "Cornelius Raymon Agustín", "Deja Jayla Bañaga", "Lalla Brielle Yabut",
  "Franklin Relano Castro" }

  main()
  DECLARE unsortedVotes, majorVoted
  SET majorVoted to totalVotes
  SEED random number generator with current system time
  FOR i is equals to 0; i less than max_size; INCREMENT i
    SET unsortedVotes to random integer modulo plus 1
  ENDFOR

  DISPLAY “Unsorted Votes”
  CALL displayVotes(unsortedVotes)
  DISPLAY candidateL[majorVoted]

  displayVotes(int arr[])
    FOR i is equals to 0; i less than max_size; INCREMENT i
      DISPLAY arr[i]
    ENDFOR

  totalVotes(int arr[], max_size)
    DECLARE “votes” with an array size of 5, “total”, “majorVotes” AS INT
    SET votes equals to 0
    SET total equals to votes
    SET majorVotes equals to 0
    CALL insertionSort(arr, max_size)
    DISPLAY “Sorted Votes”
    FOR i is equals to 0; i less than max_size; INCREMENT i
      DISPLAY arr[i]
      INCREMENT votes[candidateL - 1]
```

| | |
|---|---|
| | ENDFOR<br><br>DISPLAY “Candidates’ total votes: ”<br>FOR i is equals to 0; i less than 5; INCREMENT i<br>IF votes is greater than total<br>total is equals to votes[i]<br>majorVotes is equals to i<br>ENDIF<br>DISPLAY candidateL, votes[i]<br>RETURN majorVotes<br>ENDFOR<br>END |
| ilo_b.h (sortingAlgo.h) | START<br>insertionSort(arr, max_size)<br>DECLARE k, j, temp AS INT<br>SET k equals to 1<br>WHILE k is less than max_size<br>temp equals to arr[k]<br>j equals to k minus 1<br><br>WHILE temp is less than or equal to arr[j]<br>arr[j + 1] equals to arr[j]<br>DECREMENT j<br>END WHILE<br><br>arr[j+1] equals to temp<br>INCREMENT k<br>ENDWHILE<br>END |

## Screenshot of Algorithm Code

ilo_b.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <string>
#include "ilo_b.h"

const int max_size = 100;
void displayVotes(int arr[]);
int totalVotes(int voteArr[], const int& arrSize);

const std::string candidateL[5] = {
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustín",
    "Deja Jayla Bañaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

int main(){

    int unsortedVotes[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        unsortedVotes[i] = rand()%5 + 1;
    }

    std::cout << "Unsorted Votes:\n";
    displayVotes(unsortedVotes);
    std::cout << std::endl;

    int majorVoted = totalVotes(unsortedVotes, max_size);
    std::cout << "\nCandidate " << candidateL[majorVoted] << " is the Winner!!!";
}

void displayVotes(int arr[]){

    for (int i = 0; i < max_size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```
42
43  int totalVotes(int arr[], const int& max_size){
44
45      int votes[5] = {0};
46      int total = votes[0];
47      int majorVotes = 0;
48      insertionSort(arr, max_size);
49
50      std::cout << "Sorted Votes:\n";
51      for (int i = 0; i < max_size; i++){
52          std::cout << arr[i] << " ";
53          votes[arr[i] - 1]++;
54      }
55
56      std::cout << "\n\nCandidates' Total Votes:\n";
57      for (int i = 0; i < 5; i++){
58          if (votes[i] > total) {
59          total = votes[i];
60          majorVotes = i;
61          }
62          std::cout << candidateL[i] << " with " << votes[i] << " votes\n";
63      }
64      return majorVotes;
65  }
```

```
41  template<typename T>
42  void insertionSort(T arr[], int max_size){
43      int k=1, j, temp;
44
45      while(k < max_size){
46          temp = arr[k];
47          j = k-1;
48
49          while(temp <= arr[j]){
50              arr[j+1] = arr[j];
51              j--;
52          }
53          arr[j+1] = temp;
54          k++;
55      }
56  }
```

**Output Testing (w/ Sorted Array)**

```
Unsorted Votes:
1 3 2 5 5 3 5 3 5 1 5 3 5 3 3 1 2 3 5 4 5 2 4 4 3 4 3 1
3 1 2 4 3 5 1 3 4 3 2 2 2 2 2 4 5 5 3 5 4 1 1 5 3 1 2 3
4 3 1 4 4 2 5 5 3 2 5 2 1 3 5 1 3 5 2 1 5 2 1 2 4 1 2 3
4 2 1 4 1 5 5 1 1 5 1 1 5 3 3 3

Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Candidates Total Votes:
Bo Dalton Capistrano with 21 votes
Cornelius Raymon Agustín with 18 votes
Deja Jayla Bañaga with 24 votes
Lalla Brielle Yabut with 14 votes
Franklin Relano Castro with 23 votes

Candidate Deja Jayla Bañaga is the Winner!!!
PS D:\Downloads (HDD)\CPE010\HOA7>
```

| Index | Manual Count | Count Result of Algorithm |
|---|---|---|
| 0 | 21 Votes | 21 Votes |
| 1 | 18 Votes | 18 Votes |
| 2 | 24 Votes | 24 Votes |
| 3 | 14 Votes | 14 Votes |
| 4 | 23 Votes | 23 Votes |

| Conclusion |
| --- |
| My conclusion for this activity is I've learned the 3 common sort algorithms which are Bubble Sort, Selection Sort and Insertion Sort. Bubble Sort compares adjacent elements and swaps them if the first is greater than the second, continuing until no more swaps are needed. Selection Sort finds the smallest element in the array and swaps it with the first element, looping until all elements are in order. Insertion Sort compares each new element with already sorted elements, placing it in the correct position, and continues until the entire array is ordered. I hope these algorithms can help me with my activities in the near future to tackle any difficult problems that I may face, making it easier to organize all unsorted elements in an array. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Google AI Overview, QuillBot**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Pseudocode Translation and Summarization**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Highlighted pseudocode represents the code translation of "srand(time(0))"**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.